

CO2039: Advanced Programming



Lambda Calculus An introduction

Learning objectives

After this lecture, students will be able to:

- Explain the historical motivation of Lambda Calculus
- Identify the three syntactic forms: variable, abstraction, application
- Perform beta-reduction to evaluate lambda expressions
- Encode Booleans, numerals, pairs, and lists using Church encoding
- Construct arithmetic operations (SUCC, SUM, PROD, PRED, SUB)

Computing without machines

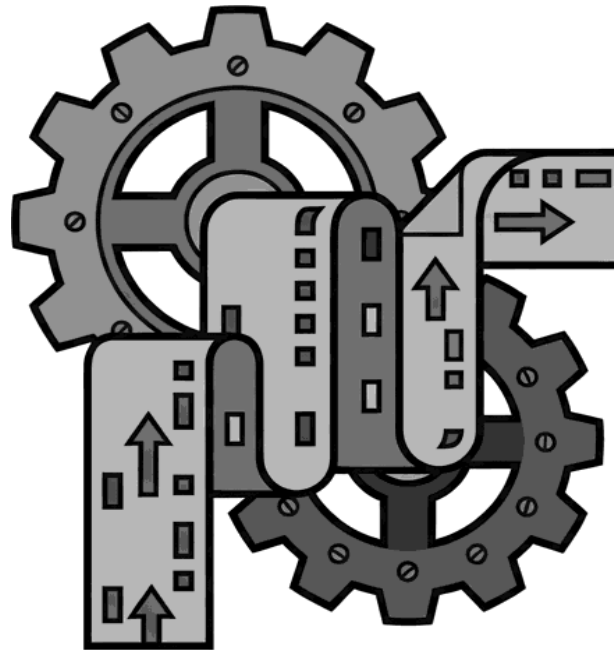
 λ 

In 1936, before silicon chips or hard drives existed, Alonzo Church asked a fundamental question: What is calculation?

He stripped away the machine to reveal the math.

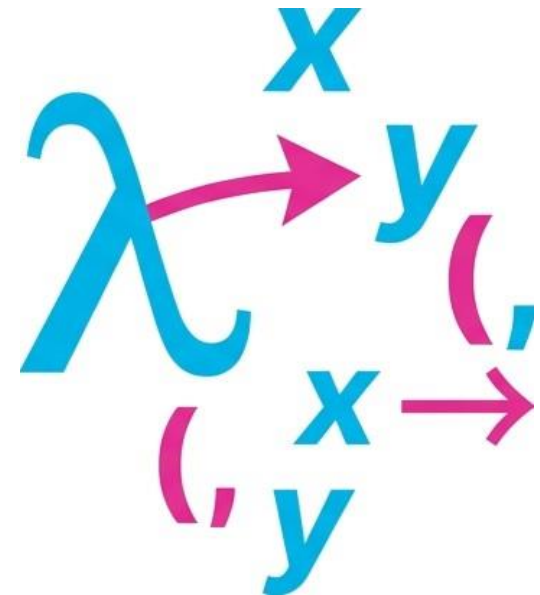
This is a purely symbolic model. No tape. No memory states. No hardware physics.

The old world



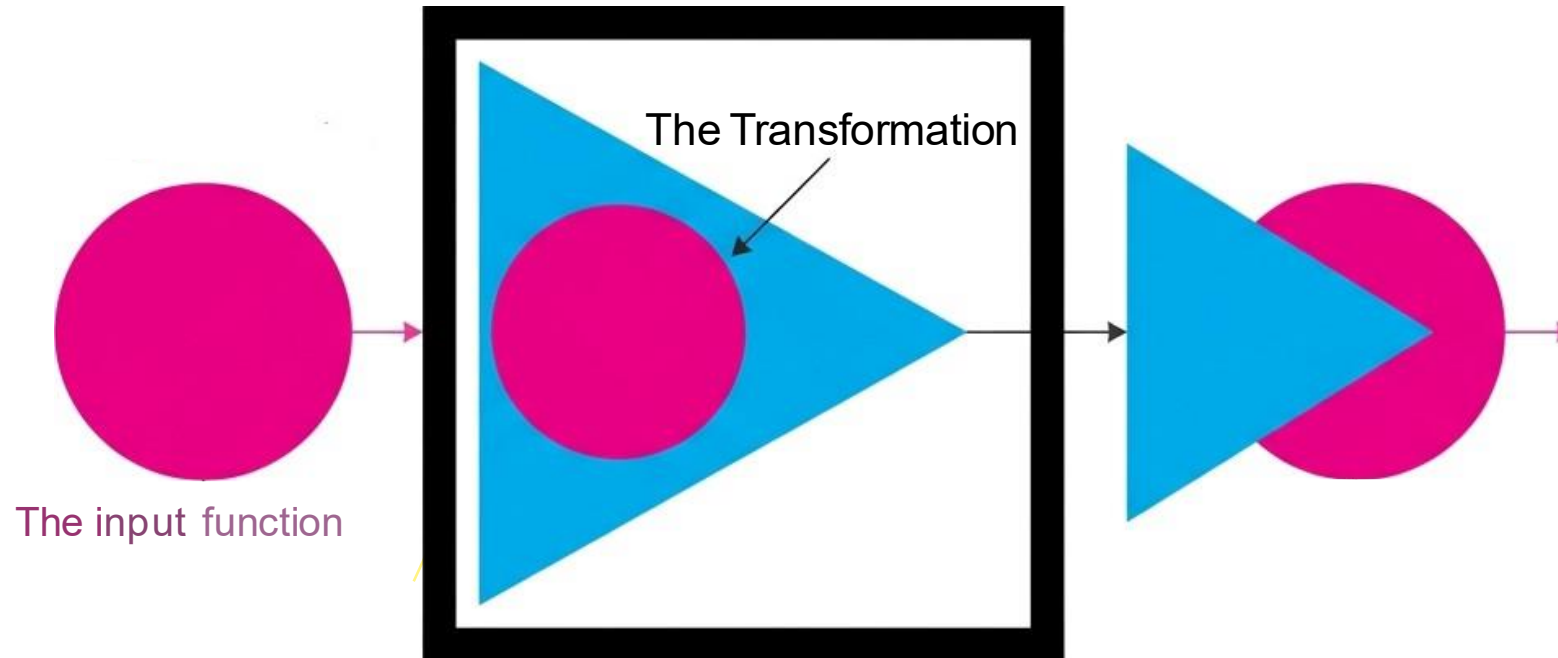
1936: The Machine (Physical)

The Lambda world



1936: The Calculus (Symbolic)

A universe of pure functions



In this model, everything is a function. Functions take other functions as input and return them as output.

Computation is simply function transformation.

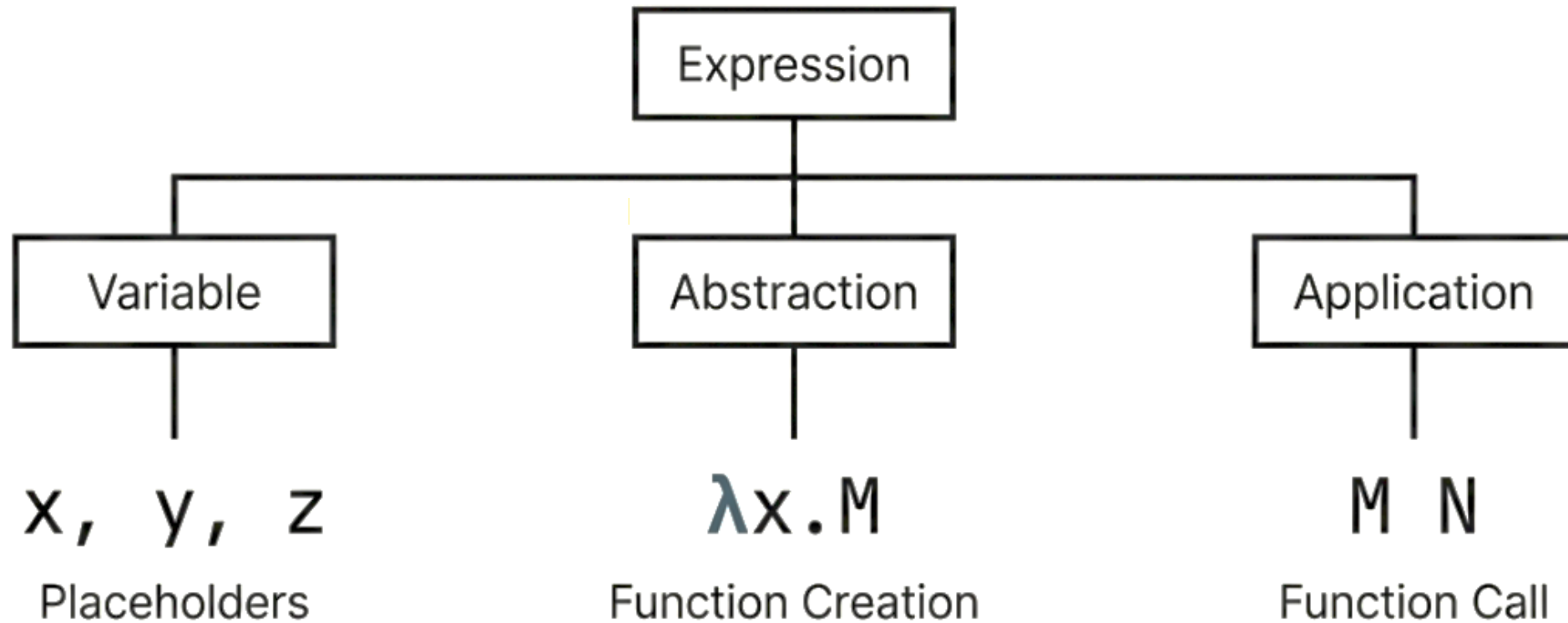
$\lambda f \lambda x f(f(x))$

Applies f twice

A function is 'higher-order' if it:

1. Takes a function as input.
2. Returns a function as output.

The three atoms of syntax



No numbers. No Booleans. No loops. Just these three rules

Scope and identity

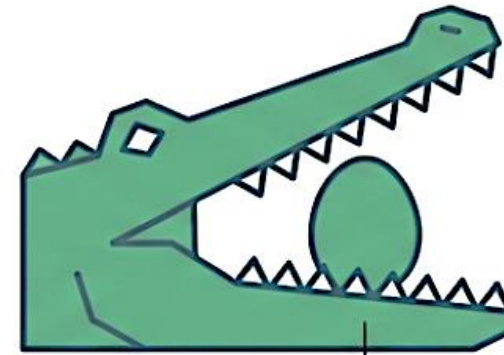
Alpha Conversion

$$\lambda x . x \equiv \lambda y . y$$

Names are arbitrary.
Structure is absolute.

Free vs. Bound

$$(\lambda x . x y)$$



Bound Variable



Free Variable

Check your understanding

$\lambda x.x$

Identity. x is Bound.

$\lambda x.y$

Constant. y is Free.

$(\lambda x.x) x$

Trick! First x is Bound, last x is Free.

The Capture problem (Shadowing)

$$\lambda x. (x + y)$$


Attempt to rename x to y

$$\lambda y. (y + y)$$


Meaning Changed!

The free variable y was captured by the new argument name. This is illegal.

The engine: Beta reduction

Computation via Substitution

$$(\lambda x. M) N \rightarrow M[x := N]$$

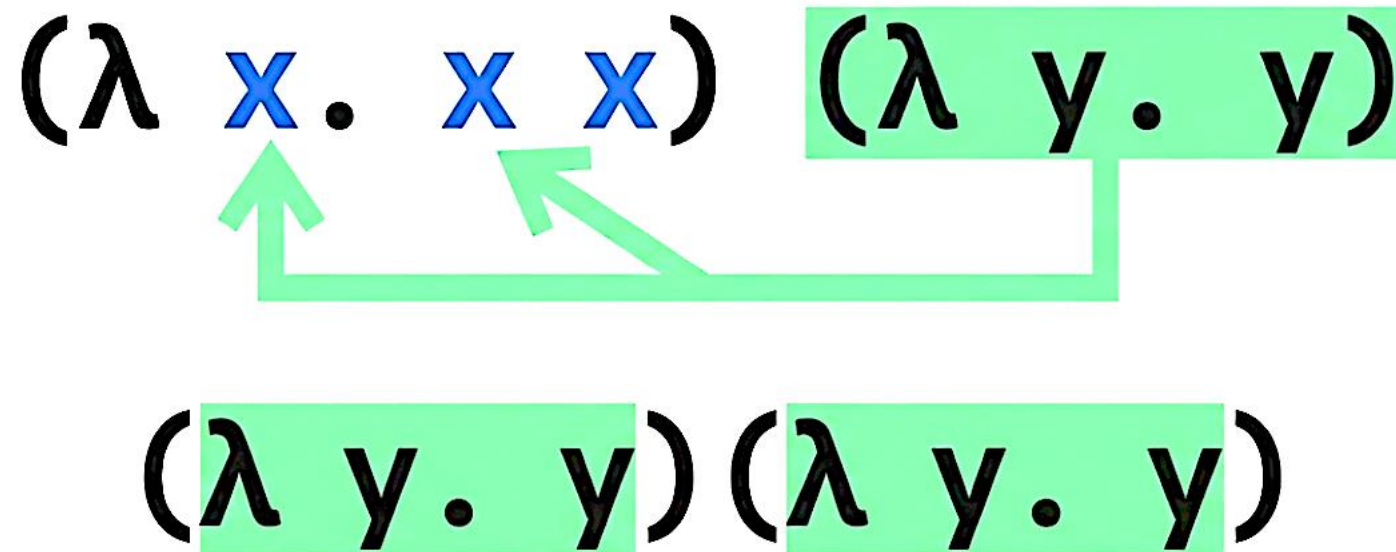
Replace every free occurrence of x in M with N

$$(\lambda x. x + 1) 5$$

↓ substitute $x := 5$

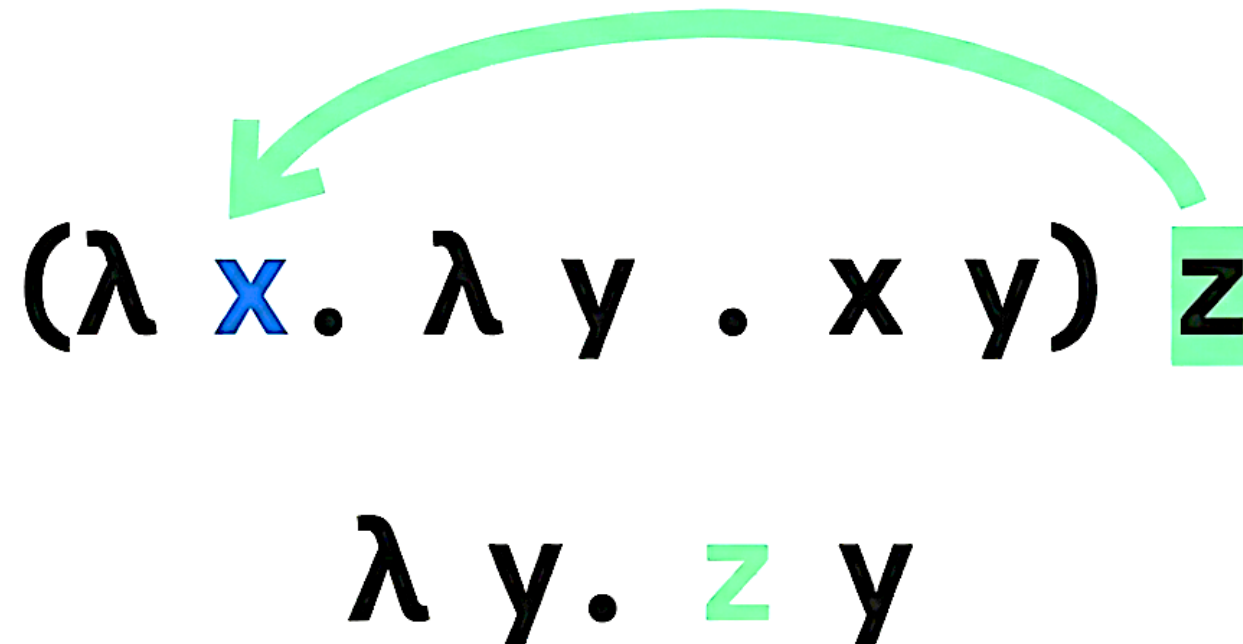
$$5 + 1$$

The engine: Beta reduction



The argument replaces every bound occurrence in the body

The danger zone: Nested scopes



The argument penetrates the inner scope to find its matching variables

Python

```
def f(x):  
    return x + 2
```

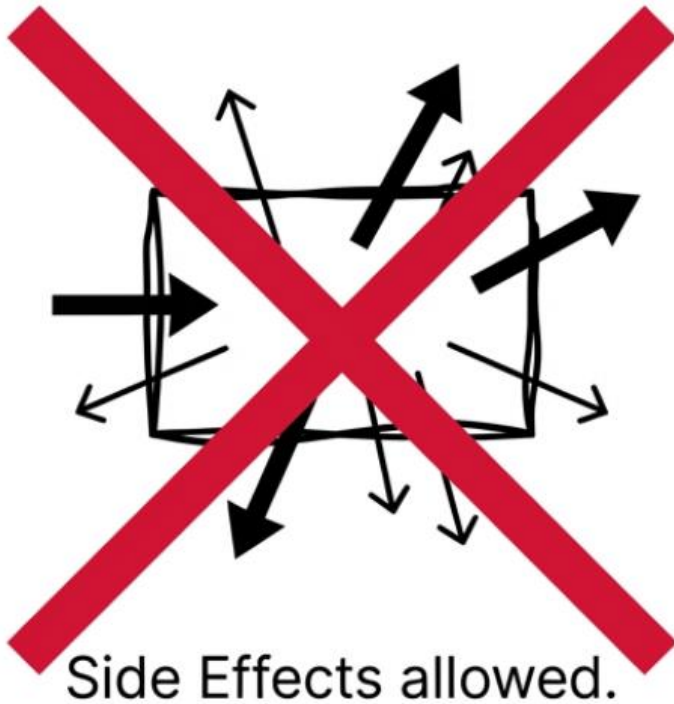
Lambda Calculus

= $\lambda x. x + 2$

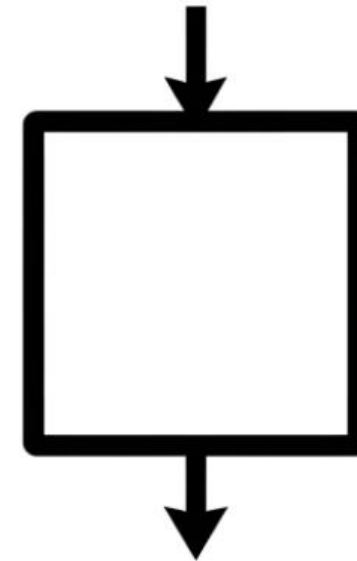
Your compiler translate the left into the right

The clean room protocol

IMPERATIVE / STATEFUL



PURE FUNCTIONAL



Referential Transparency.

If $f(x) = y$, then $f(x)$ can always be replaced by y .

The Combinator toolset

Identity (I)

Lambda: $\lambda x.x$

JS: $x \Rightarrow x$

Returns the argument.

Mockingbird (M)

Lambda: $\lambda x.xx$

JS: $x \Rightarrow x(x)$

Self-application.

Kestrel (K)

Lambda: $\lambda x.\lambda y.x$

JS: $x \Rightarrow y \Rightarrow x$

Returns the first.

Kite (KI)

Lambda: $\lambda x.\lambda y.y$

JS: $x \Rightarrow y \Rightarrow y$

Returns the second.

A combinator is a function with no free variables. Totally self-contained.

The crisis of data

~~Integers~~

~~Booleans~~

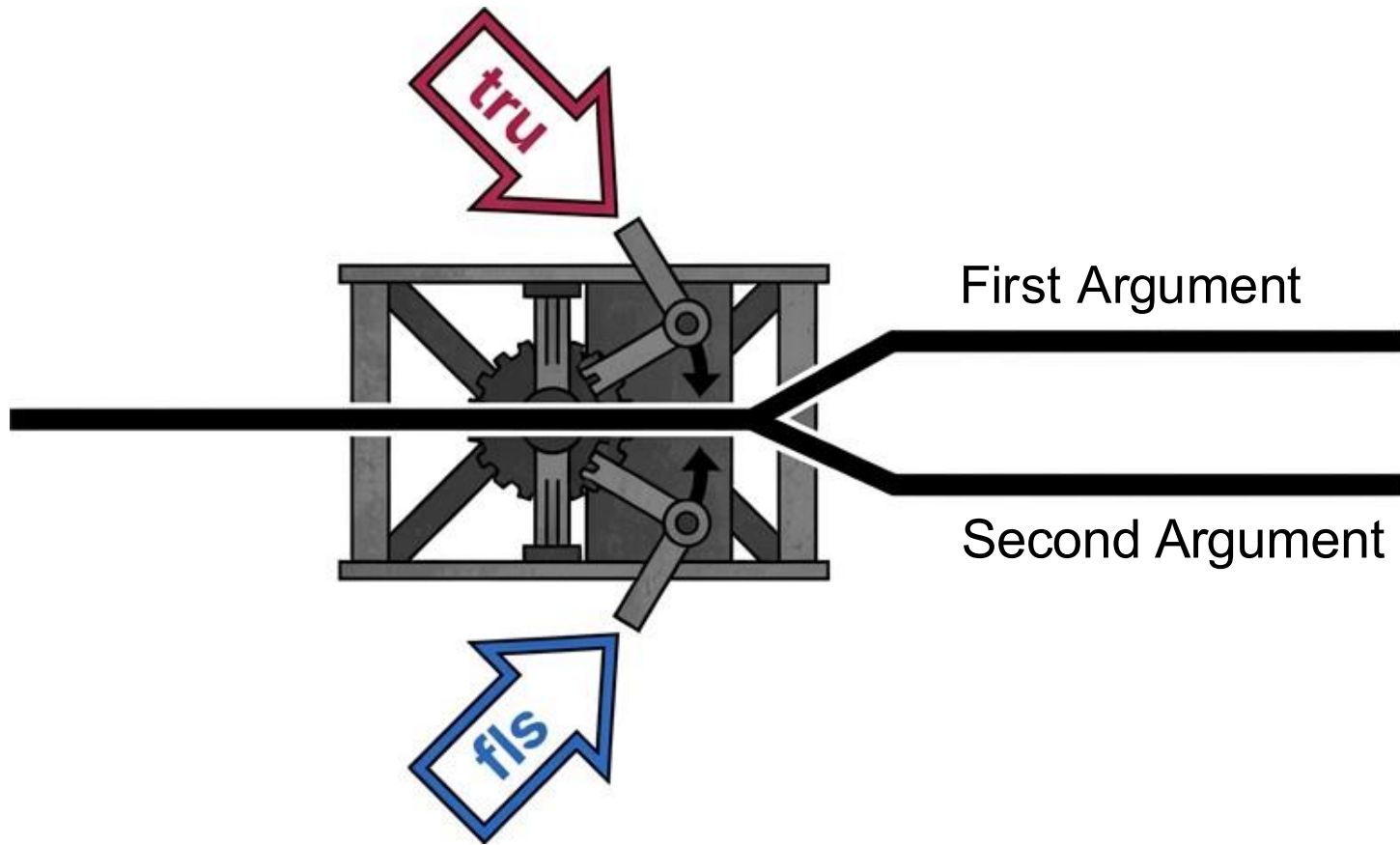
~~Operators (+, -, ==)~~

~~Control Structures (if, while)~~

The solution: Church Encoding.

“We don’t represent data; we represent behavior”

Truth is a choice



$$\text{tru} = \lambda t. \lambda f. t$$

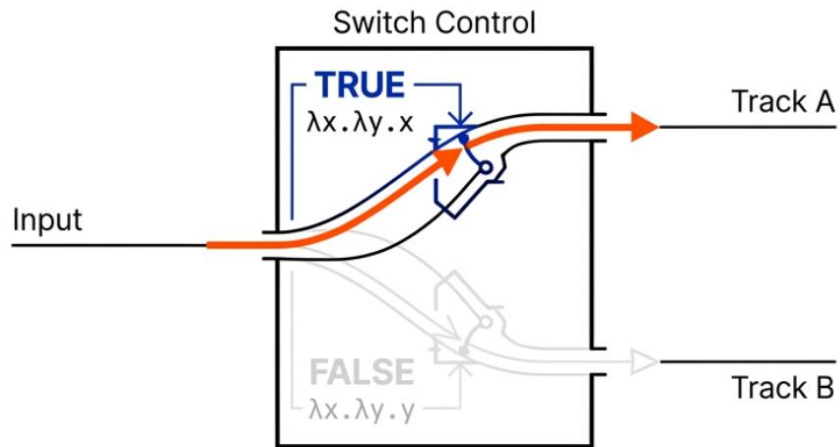
$$\text{fls} = \lambda t. \lambda f. f$$

Booleans are not values. They are functions that choose.

Control flow without keywords

The Test

`test = λb.λthen.λelse.b then else`



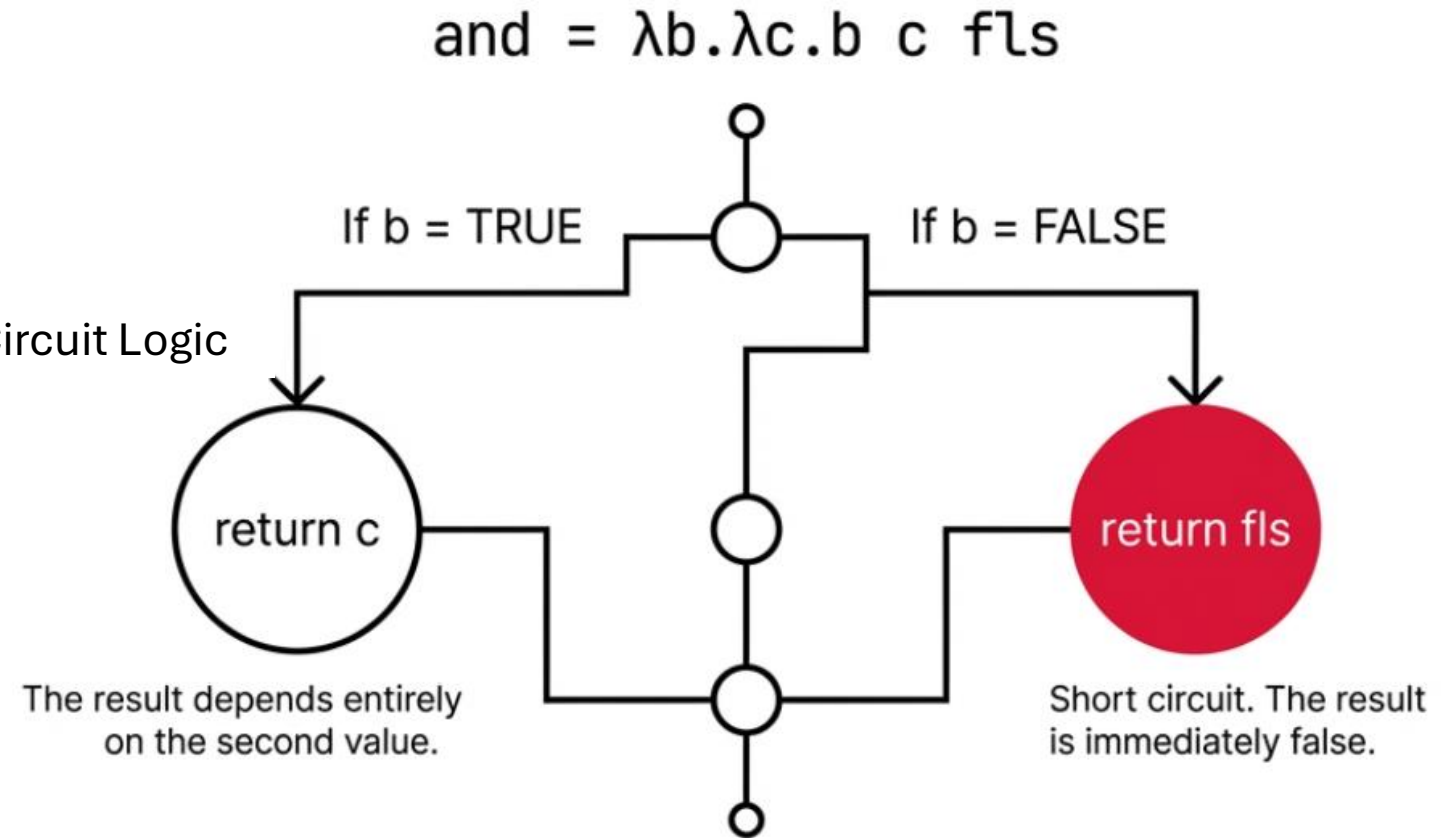
Definition	<code>test = λb.λthen.λelse.b then else</code>
The Call	<code>test tru A B</code>
Expansion	<code>(λb.b A B) tru</code>
Application	<code>tru A B</code>
Definition Expansion	<code>(λt.λf.t) A B</code>
Selection	A

Building logic gates

$\text{and} = \lambda b.\lambda c.b\ c\ \text{fls}$

$\text{and}\ \text{tru}\ \text{tru} \rightarrow$	tru
$\text{and}\ \text{tru}\ \text{fls} \rightarrow$	fls
$\text{and}\ \text{fls}\ \text{tru} \rightarrow$	fls
$\text{and}\ \text{fls}\ \text{fls} \rightarrow$	fls

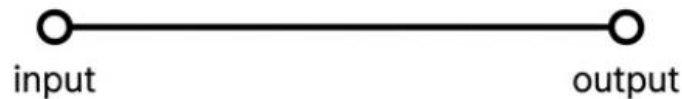
Short Circuit Logic



Encoding arithmetic: Church numerals

Numbers are not magnitudes. They are frequencies.

0

$$\lambda f. \lambda x. x$$


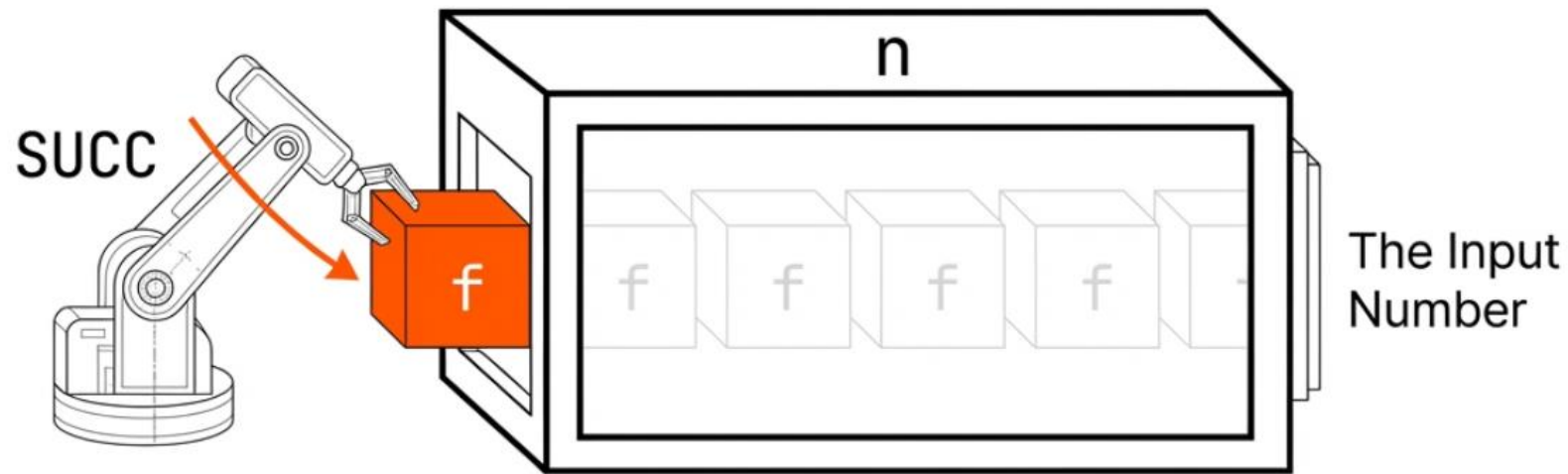
1

$$\lambda f. \lambda x. f \ x$$


2

$$\lambda f. \lambda x. f \ (f \ x)$$


The Successor function

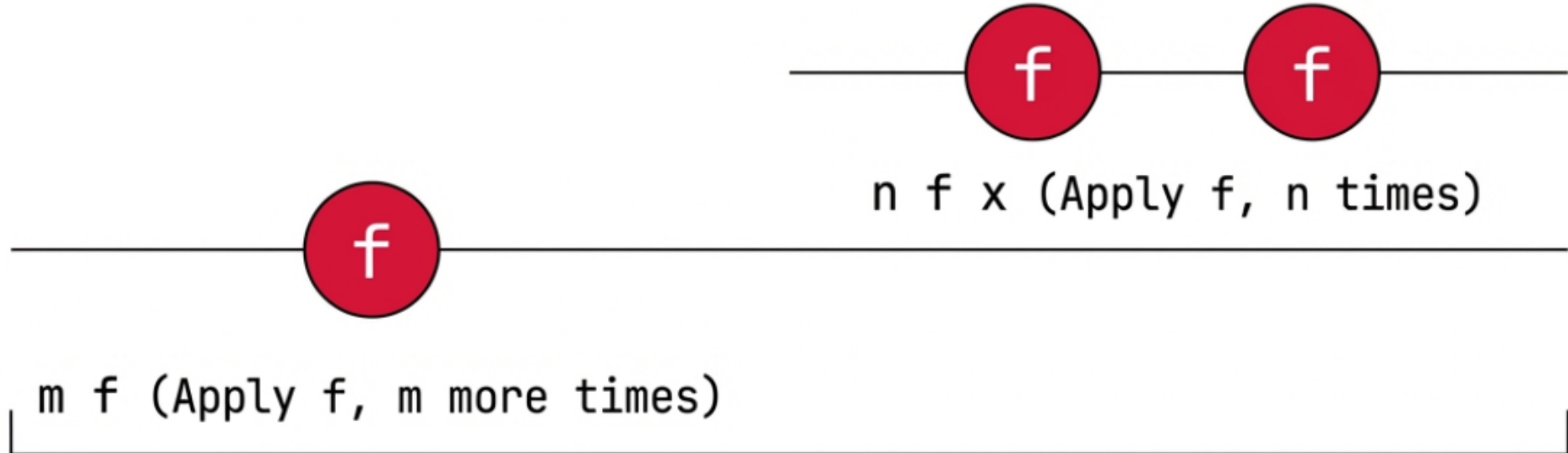


$$\text{SUCC} = \lambda n. \lambda f. \lambda x. \textcolor{orange}{f} (n f x)$$

To compute $n+1$, we simply wrap one more layer of function application around the existing number.

Addition (Sum)

$SUM = \lambda m. \lambda n. \lambda f. \lambda x. m \ f \ (n \ f \ x)$



$$f \ (f \ (f \ x)) = 3$$

Multiplication (Prod)

$\text{PROD} = \lambda m. \lambda n. \lambda f. \lambda x. m \text{ (n f) } x$



ITERATION (Linear)



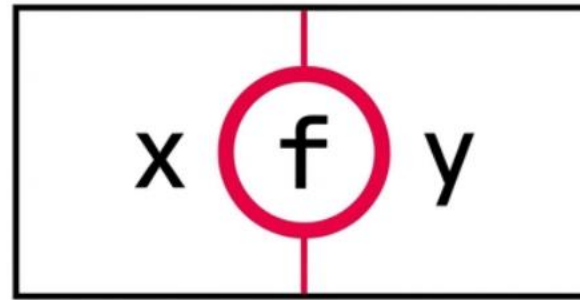
COMPOSITION (Nested)

Addition runs the loop M time, then N times.

Multiplication runs the 'Do N times' loop, M times.

The Pair

Data Structures as Closures



PAIR = $\lambda x. \lambda y. \lambda f. f \ x \ y$

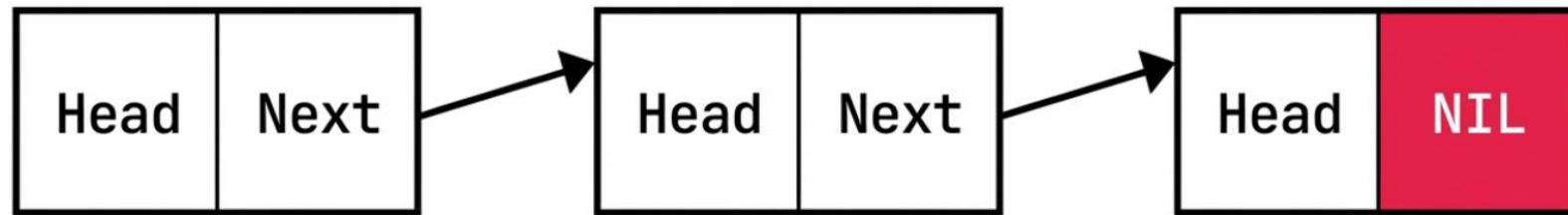
FIRST = $\lambda p. p \ \text{TRUE}$

SECOND = $\lambda p. p \ \text{FALSE}$

FIRST (PAIR A B) $\longrightarrow$ A

The Pair holds the values hostage until a selector (Boolean) releases one.

List & Null states

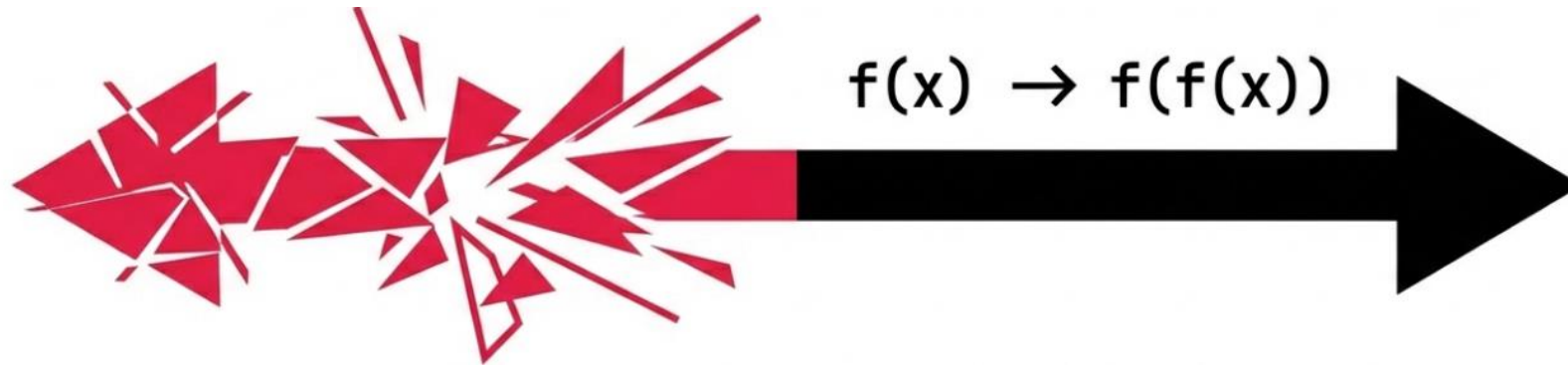


$NIL = \lambda x. TRUE$

$ISEMPTY = \lambda l. l (\lambda h. \lambda t. FALSE) TRUE$

$ISEMPTY\ NIL \rightarrow TRUE$

The Predecessor problem

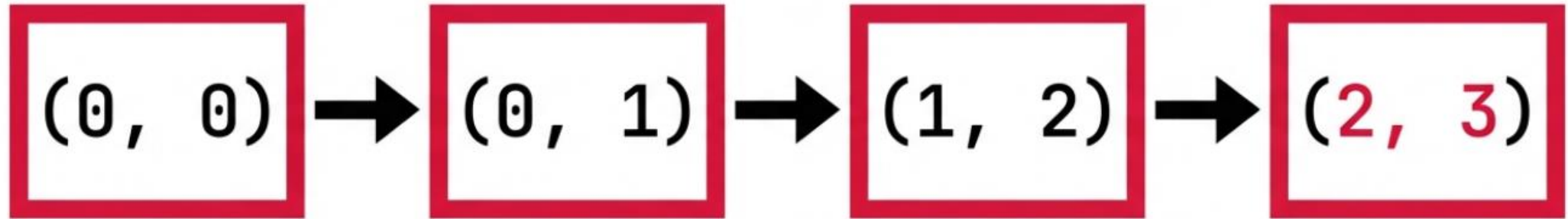


Computing $N-1$ is the hardest problem in Lambda Calculus.

Why? Function application is destructive.

Given ' $f(f(x))$ ', you cannot look inside to find ' $f(x)$ '.

Solution: The sliding window



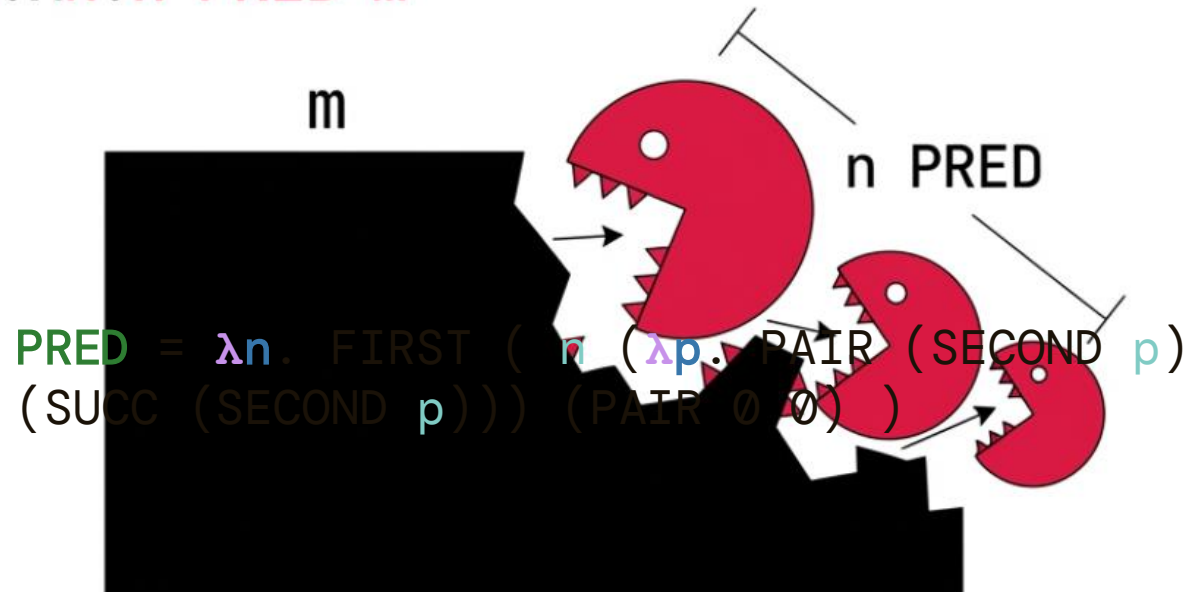
To find PRED 3, we count up to 3, but we track the previous number in a Pair.

Result: Take FIRST of (2,3) -> 2

PRED = $\lambda n. \text{FIRST} (n (\lambda p. \text{PAIR} (\text{SECOND } p) (\text{SUCC} (\text{SECOND } p)))) (\text{PAIR } 0 \ 0))$

Subtraction

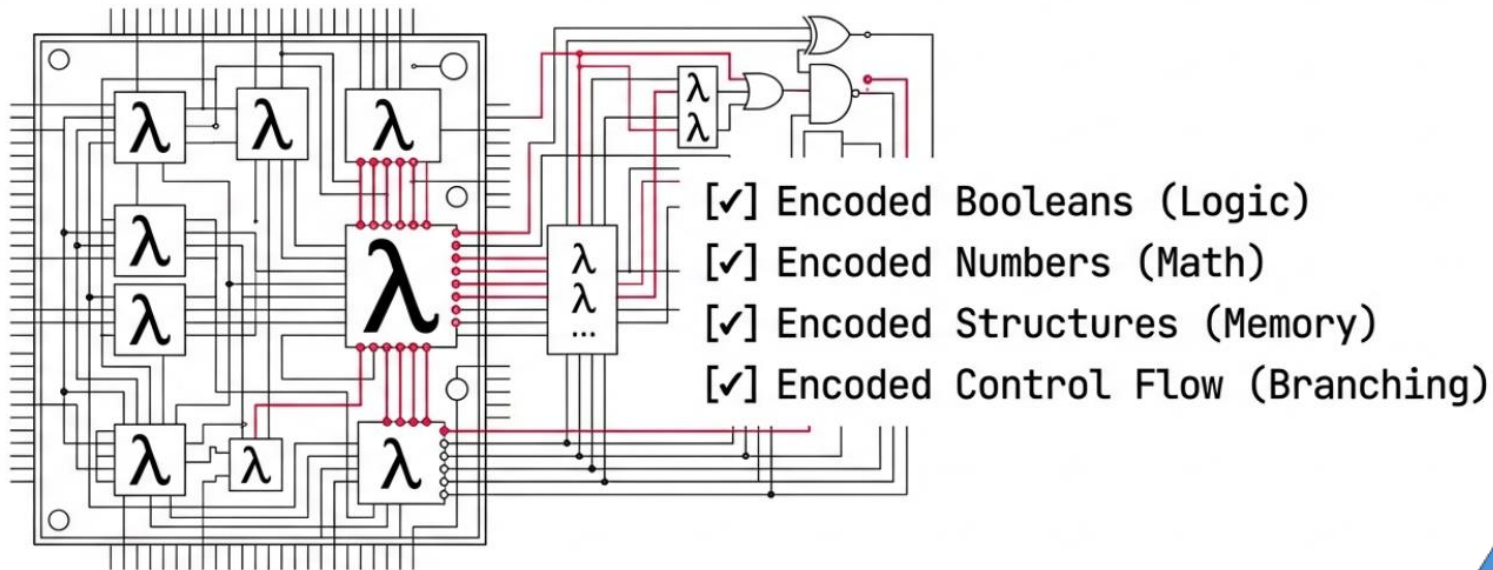
SUBTRACT = $\lambda m. \lambda n. n \text{ PRED } m$



1. Take number m .
2. Apply the PRED function n times.

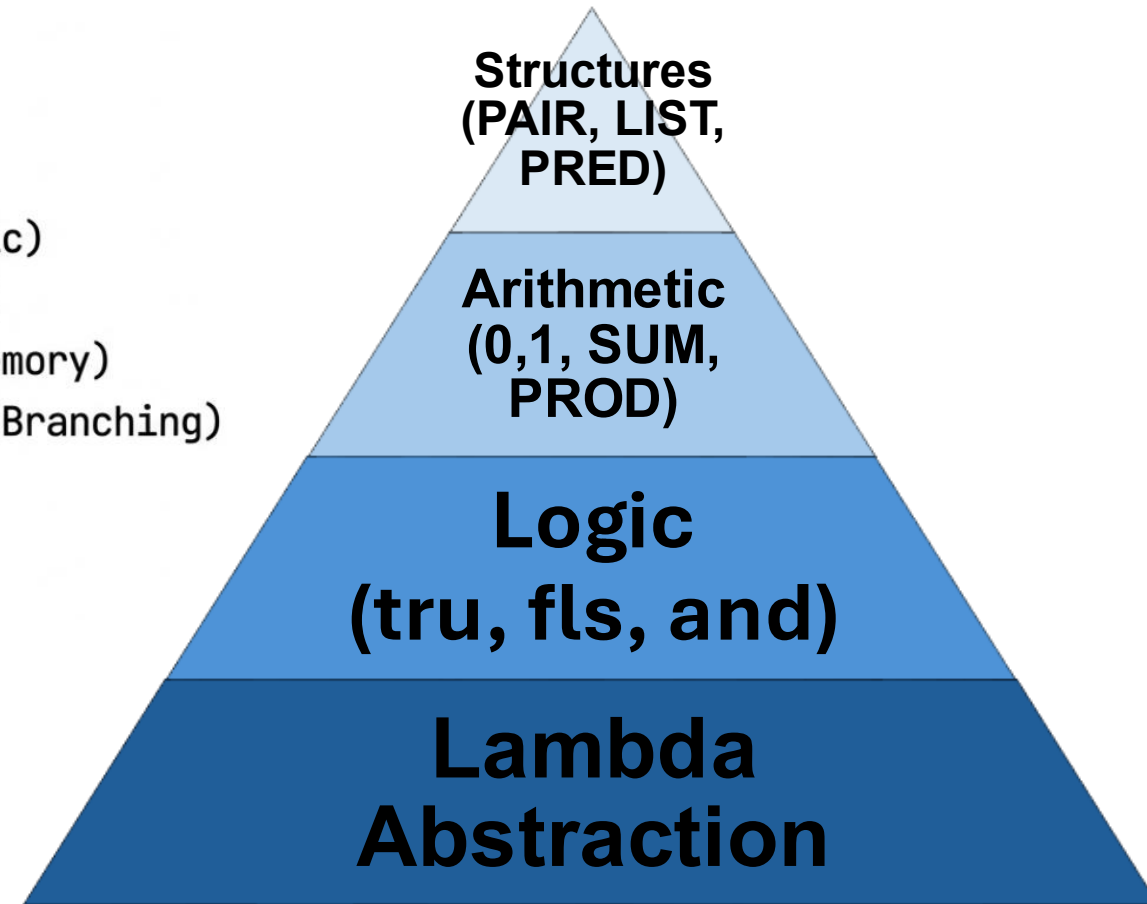
We have successfully rebuilt arithmetic without any hardware support

The big insight



We require no built-in types.

The Lambda Calculus is self-contained.



Takeaway messages

Computation is just function transformation

From the void, we built the universe:

Logic, Arithmetic, Data structures, Control flow

Next lecture: Recursion, typed lambda calculus, Church-Turing thesis